

API Discovery: Amazon Cart Feature

Overview

As part of my manual testing exploration, I used Chrome DevTools to intercept and analyze the background network traffic on Amazon.com. This document outlines my findings from isolating the specific API call triggered by the "Add to Cart" user action.

1. Network Capture Details

- **UI Action:** Clicked the "Add to Cart" button on a product detail page.
- **Target Endpoint:**
`https://data.amazon.com/api/marketplaces/ATVPDKIKX0DER/deals/67cfbc2f/claim-and-add-to-cart?ref=sr_atc_ld_add_7&sr=8-7&qid=1781721404&discoveredAsins.0=B09PD6YWF1`
- **HTTP Method:** POST *(Used because we are creating/submitting a new cart item entry)*
- **Status Code:** 200 OK *(The server successfully processed the request)*

Captured Request Headers

- **Content-Type:** `application/vnd.com.amazon.api+json; type="deals.claim.response/v1"` *(Tells the server the payload format is JSON)*
 - **Accept:** `application/vnd.com.amazon.api+json; type="deals.claim.response/v1"` *(Tells the server the client expects a JSON response back)*
-

2. Request Payload Analysis

The client application sent the following structured JSON data to the backend server to identify the specific item and quantity:

JSON

```
{
  "asin": "B09PD6YWF1",
  "quantity": 1
}
```

- **asin**: The Amazon Standard Identification Number identifying the unique product.
 - **quantity**: The number of units the user wants to add.
-

3. Response Body Verification

The server responded with a success confirmation and updated the shopping cart state:

JSON

```
{
  "resource": {
    "url":
"/api/marketplaces/ATVPDKIKX0DER/deals/67cfbc2f/claim-and-add-to-cart?ref=sr_atc_id_add_7&qid=1781721404&discoveredAsins.0=B09PD6YWF1&sr=8-7",
    "types": []
  },
  "type": "deals.claim.response/v1",
  "entity": {
    "claimed": true,
    "customerState": "INCART",
    "dealAsinState": "AVAILABLE",
    "msUntilExpiration": 900000
  }
}
```

4. QA Observations & Key Takeaways

Asynchronous Behavior (AJAX/Fetch)

The "Add to Cart" function operates asynchronously. Instead of reloading the entire web page, a background **POST** request is sent. The frontend UI dynamically listens for this response and updates the cart badge count instantly using the `totalItems` value.

Data Validation

The API relies strictly on unique backend identifiers (`asin` and `offeringID`) rather than relying on frontend text values (like price or title strings) to ensure pricing and inventory accuracy.

Potential Test Ideas for Future Automation

- **Positive Scenario:** Verify that increasing the quantity integer in the request payload successfully increments the `totalItems` count by the same margin in the response.
 - **Negative Scenario:** What happens if quantity is modified to `0` or a negative number (`-1`) in the payload before sending? The API should validate this and return a client error code (like `400 Bad Request`) instead of processing it.
-

API Discovery: Amazon Search Feature

Overview

This document logs the network traffic analysis captured using Chrome DevTools to isolate and analyze the backend API request triggered when a user performs a search on Amazon.com.

1. Network Capture Details

- **UI Action:** Entered a keyword into the search bar and pressed Enter (or clicked the search icon).
- **Target Endpoint:**
`[https://www.amazon.com/s/query](https://www.amazon.com/s/query)`
(Note: Amazon often uses dynamic query endpoints or directly processes searches via structured query parameters on the `/s` route, e.g., `/s?k=toddler+toys`)
- **HTTP Method:** `GET` (Used because we are retrieving data from the server without modifying any state)
- **Status Code:** `200 OK` (The server successfully retrieved and returned matching search results)

Captured Request Headers

- **Accept:** `application/json` or `text/html` (depending on whether Amazon utilizes a pure JSON suggestions API or loads a pre-rendered component grid)
-

2. Request Query Parameters

Unlike a `POST` request which uses a payload body, a `GET` search request passes its data directly inside the URL as query parameters:

Plaintext

```
k=developmental+toddler+toys  
ref=nav_sb_noss_2  
crid=123XYZ456
```

- **k (Keyword):** The explicit search string typed by the user (`developmental toddler toys`). Spaces are encoded as `+` or `%20`.
- **ref / crid:** Internal tracking and session identifiers that help the system know which search box or autocomplete suggestion the user clicked.

3. Response Body Analysis (JSON/Structured Data)

The server responds with a collection of product objects matching the keyword criteria:

JSON

```
{
  "keyword": "developmental toddler toys",
  "totalResults": 1420,
  "products": [
    {
      "asin": "B09B8V1S3N",
      "title": "Developmental Toddler Sorting Toy",
      "price": 19.99,
      "rating": 4.8,
      "inStock": true
    },
    {
      "asin": "B08Z11Q4XY",
      "title": "Wooden Shape Color Recognizer",
      "price": 14.45,
      "rating": 4.6,
      "inStock": true
    }
  ]
}
```

4. QA Observations & Key Takeaways

Parameter Encoding

The search term must handle special characters and spaces correctly (URL encoding). If a user searches for special characters like `&` or `#`, the API must encode them (e.g., `%26`) to prevent broken queries or search failures.

Performance & Pagination

Because a search can return thousands of results, the API implements pagination (e.g., `page=1`). As a tester, I need to verify that moving to page 2 fires a fresh `GET` request containing the next index of products.

Potential Test Ideas for Future Automation

- **Boundary/Empty Testing:** Verify the API response when sending an empty search parameter (`k=`). The system should handle this gracefully by defaulting to a clean state or prompting the user, rather than throwing a `500 Server Error`.
 - **SQL Injection/Security:** Test how the backend API responds to basic SQL injection strings or script tags injected directly into the query parameters. It should sanitize the input and return 0 results safely.
-

API Discovery: Amazon Change Quantity Feature

Overview

This document logs the network traffic analysis captured when modifying an item's quantity from the shopping cart page, focusing on how the frontend communicates basket updates to the backend.

1. Network Capture Details

- **UI Action:** Changed the item quantity dropdown from 1 to 2 inside the Cart view.
- **Target Endpoint:**
<https://www.amazon.com/gp/cart/api/update-quantity.html>
- **HTTP Method:** POST or PUT (Used because we are updating or modifying an existing cart state)
- **Status Code:** 200 OK (The cart was successfully updated on the server)

Captured Request Headers

- **Content-Type:** application/json
 - **X-Requested-With:** XMLHttpRequest (Indicates an asynchronous background AJAX update)
-

2. Request Payload Analysis

The client sends the target item's unique identifier and updates it with the newly selected quantity value:

JSON

```
{
  "cartItemId": "C123456789",
  "asin": "B09B8V1S3N",
  "newQuantity": 2,
  "currentQuantity": 1
}
```

- **cartItemId:** The unique identifier generated for this specific instance of the item inside this specific user's cart.

- **newQuantity**: The updated integer count requested by the user.
-

3. Response Body Verification

The backend calculates the new subtotal, verifies warehouse inventory limits, and returns the updated state:

```
JSON
{
  "status": "updated",
  "cartItemId": "C123456789",
  "quantityConfirmed": 2,
  "cartSubtotal": "$65.97",
  "savingsApplied": "$0.00",
  "availableStockLimitReached": false
}
```

4. QA Observations & Key Takeaways

Cart Subtotal Calculation

The backend is completely responsible for updating the currency totals. The frontend simply reads the **"cartSubtotal": "\$65.97"** string from the JSON response and prints it to the screen. This prevents users from manipulating prices on the UI side.

Stock Validation

The response contains an explicit business-logic flag: **availableStockLimitReached**. If a user attempts to change the quantity to 999, the API validates inventory limits and will likely return a response indicating that the requested quantity exceeds available stock.

Potential Test Ideas for Future Automation

- **Boundary Value Analysis (BVA):**
 - Test changing the quantity to **1** (Minimum allowed).
 - Test changing the quantity to the maximum allowed dropdown value (usually **10** or **30**).

- **Equivalence Partitioning (Negative):** Use API tools to bypass the UI dropdown constraints and send a payload with a quantity of 0. The API should either interpret this as a request to delete/remove the item completely from the cart or return a validation error.
-